# Python for Data Engineering

# Lesson Program

## Learning Objectives

1. Understand what Data Engineers do in organizations
2. Differentiate Engineers, Analysts, and Data Scientists
3. Learn core system concepts like OLTP, OLAP, Batch and Streaming
4. Apply Python best practices for building data systems

**PART 1- THEORY (60 MINS)**

## 1. Introduction to Data Roles

**Data Engineer vs Data Analyst vs Data Scientist**

**Data Engineers**

- Build and maintain data pipelines and infrastructure
- Ensure data is reliable, scalable, and accessible
- Work with databases, ETL processes, and distributed systems

**Data Analysts**

- Analyze structured data to generate insights
- Create dashboards, reports, and visualizations
- Use SQL, Excel, and BI tools

**Data Scientists**

- Build predictive models and machine learning systems
- Work with statistics, experimentation, and advanced analytics
- Use Python/R, ML libraries, and data modeling techniques

## 2. OLTP vs OLAP Systems

**OLTP (Online Transaction Processing)**

- Handles real-time transactional data
- Example: banking systems, e-commerce checkouts
- Optimized for fast reads/writes (many small queries)

**OLAP (Online Analytical Processing)**

- Handles analytical queries on large datasets
- Example: data warehouses
- Optimized for complex queries and aggregations

## 3. Batch vs Streaming Data

**Batch Processing**

- Processes data in chunks at scheduled intervals
- Example: daily sales reports
- Tools: Airflow, Spark

**Streaming Processing**

- Processes data in real-time as it arrives
- Example: live user activity tracking
- Tools: Kafka, Flink

## 4. Data Lifecycle

**Stages:**

1. **Ingestion**

   Collecting data from sources (APIs, databases, logs)

2. **Storage**

   Saving raw or processed data (data lakes, warehouses)

3. **Transformation**

   Cleaning and structuring data (ETL/ELT)

4. **Serving**

   Making data available for analytics or applications

## 5. Data Engineering in Different Organizations

**Product-Focused Data Engineering**

- Supports real-time applications
- Focus on performance and uptime
- Example: recommendation systems

**Analytics-Focused Data Engineering**

- Supports business intelligence
- Focus on clean, reliable datasets
- Example: dashboards and reporting systems

## 6. Architecture Diagrams

Tools for designing data systems:

- Lucidchart
- Draw.io (diagrams.net)
- Miro

**Key Components to Diagram:**

- Data sources
- Pipelines
- Storage layers
- Processing engines
- End users

## PART 2- PRACTICAL (90 MINS)

## 7. Python Fundamentals for Data Engineering

**Core Concepts**

- Variables and data types
- Lists, dictionaries, sets
- Functions
- Loops and conditionals

**Example**

```
def process_data(data):
```

```
    return [x * 2 for x in data if x > 0]
```

**8. Writing Modular and Testable Code**

**Best Practices**

- Break code into small functions
- Use clear naming conventions
- Avoid hardcoding values
- Write unit tests

**Example**

```
def clean_data(record):
    return record.strip().lower()


def transform_data(records):
    return [clean_data(r) for r in records]
```

**9. Working with File Formats**

**CSV**

```
import csv


with open('data.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

**JSON**

```
import json
with open('data.json') as f:
    data = json.load(f)
```

**Parquet (using pandas)**

```
import pandas as pd
```

```
df = pd.read_parquet('data.parquet')
```

**Avro (using fastavro)**

```
from fastavro import reader

with open('data.avro', 'rb') as f:
    for record in reader(f):
        print(record)
```

**10. Error Handling and Logging**

**Error Handling**

```
try:
    value = int("abc")
except ValueError as e:
    print("Conversion failed:", e)
```

**Logging**

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Pipeline started")
logging.error("An error occurred")
```

**11. Summary**

By the end of this lesson, students would:

- Understand key data roles and systems
- Know how data flows through a lifecycle
- Be comfortable with Python basics
- Handle different data formats
- Write clean, modular, and reliable code